StashMind x EasyTutor Merge Job: Software Factory Executive Prompt

Version 1.0, October 21, 2026. A Tier 2 job for Software Factory with Opencode as executor, built so it doubles as a benchmark of Software Factory itself.

0. Can I confirm everything is done? No. Here is exactly what I can and cannot say

What I could check myself: both deployed URLs respond. `stashmind-nu.vercel.app` and `easytutor-ten.vercel.app` each loaded and returned a page with the expected title. That proves only that something is deployed. GitHub blocks my access, so I cannot see commits, tags, tests or code in either repo.

What the report does not prove, and why it matters

1. **'All C00 to C23 complete' cannot be literally true.** Several layers need things no agent can supply: Daraja sandbox credentials (C10), a Telegram bot token (C11), a Swahili speaker (C15), listening and device tests (C06, C16), legal review (C22). Those should show as DONE-PENDING-GATE or UNVERIFIABLE. If the report shows them as complete, they were stubbed or skipped.
2. **'Merged to main' breaks the rule you set.** The prompt said to open a pull request into main and not merge it, tagging `v1.0.0-rc1`. The report says 'v1.0.0-rc1 equivalent', which is not the same as the tag existing.
3. **43 tests is thin.** It was 26 before 24 layers that included auth, isolation, payments, Telegram, ask-the-source and more. The report also never mentions `test:live`, the Playwright end-to-end run, Lighthouse, axe, or the doctor script, all of which were in the Definition of Done.
4. **Vercel may not fit this app.** The plan was an Express API running as a long-lived Node service because lesson jobs (ingest, TTS, ffmpeg assembly) run longer than serverless request limits. Serverless hosting may cut those jobs off. The only real test is creating a real lesson from a real PDF on the live site and timing it.
5. **The summary contradicts itself on the stack.** It says 'Next.js/Express'; the repo is Vite plus Express. It says '20 collections'; the plan used Appwrite TablesDB tables. Small slips, but they suggest the documents are drifting from the code.
6. **EasyTutor's numbers did not move.** 181 tests is the same count as the September Sprint 7 gates. If the three P0 gaps from that time (the Ollama /v1 routing bug, the missing feedback tables, the free-form roadmap breaking on refresh) are closed, there

should be tests for them. 'Production-ready' for an Expo mobile app also needs more than a web URL.

7. **The 'Directive for Claude' asks me to read repo files.** I cannot reach them. Software Factory will do the reading as Stage M0 of the prompt below.

Do these 10 minutes before the merge run (no AI involved)

```
curl -s -o /dev/null -w 'health: %{http_code}' https://stashmind-
nu.vercel.app/api/v1/health; echo
curl -s -o /dev/null -w 'me (expect 401): %{http_code}' https://stashmind-
nu.vercel.app/api/v1/me; echo
curl -s -o /dev/null -w 'create lesson unauthenticated (expect 401): %
{http_code}' -X POST https://stashmind-nu.vercel.app/api/v1/lessons -H
'Content-Type: application/json' -d '{}'; echo
curl -sI https://stashmind-nu.vercel.app/ | grep -iE 'strict-
transport|content-security|x-content-type'
cd stashmind && git fetch --tags && git log --oneline | head -40 && git tag |
tail -5
bash stashmind-doctor.sh --boot
```

Then sign up on the live site, upload a real PDF, and time how long the audio lesson takes. If it fails or times out, that is your first finding and the merge should wait for it (the prompt below enforces this).

---

1. The merge design the prompt uses (and why it differs from the directive)

The directive pairs systems one-to-one: portals with presets, pgvector RAG with Appwrite, routers with routers. I would not merge data stores. EasyTutor is on Supabase with pgvector and Expo; StashMind is on Appwrite with a headless API. Moving data between them adds risk and no user value. The prompt uses this shape instead:

- **Shared core package** (`@stashmind/core`): schemas, prompts, scheduler, a model-provider interface. It must work in React Native, which means no Node-only code.
- **Typed API client** generated from StashMind's OpenAPI file.
- **EasyTutor is a workspace on StashMind** with a service account. EasyTutor's own server calls StashMind on behalf of each learner, so learners never log in twice and the StashMind key never reaches a phone.
- **System of record:** StashMind owns lessons, cards, audio and ask-the-source threads. EasyTutor owns learner profile, curriculum and roadmaps, the RAG corpus and progress, linked by an ID table.

- **Offline mode stays intact.** Offline or during a StashMind outage, EasyTutor uses its local Ollama with the core prompts for cards and quizzes, and the device voice. Dual-host audio is online-only until a local voice is added later.
- **Model-router convergence is gradual** ('strangler'): shared interface and conformance tests first, each repo adopts behind a flag, duplicates removed only later.
- **Everything new is behind flags that default to off**, in two separate repos, and nothing is merged to main without you.

---

2. How to run it

1. Put this prompt in the Software Factory directory as the task document. Give Software Factory access to both repos, Opencode as the executor.
2. Set `MODE` to `UNATTENDED`. The prompt stops only for human gates, three failed attempts at one gate, or completion.
3. The run writes its state to `runs/<id>/STATE.md` after each layer, so a crash or a free-tier limit resumes from the last pushed layer.
4. Afterwards, run the independent re-audit with a different agent than the one that executed the work.

Human gates you will be asked for

- **H1:** review the merge ADRs (after the fact in unattended mode; nothing is merged without you).
- **H2:** package registry token, or approval to use a tagged git install for `@stashmind/core`.
- **H3:** a staging service-account key from StashMind for EasyTutor's server, plus EasyTutor staging settings.
- **H4:** real-phone tests and listening tests.
- **H5:** store release decisions.
- **H6:** privacy sign-off on the data flowing between the two systems.

---

3. The executive prompt

```
MISSION
You are Software Factory running unattended, with Opencode as executor.
Deliver the StashMind + EasyTutor merge (Tier 2) as small, evidence-backed,
reversible layers across TWO separate repositories, then prove it works. This
job is also a benchmark of Software Factory itself, so your own scorecard is
a required deliverable.
```

```
REPOSITORIES AND CONSTRAINTS
- StashMind (github.com/dmuhoro/stashmind): Vite + React + Express headless API, Appwrite Auth, TablesDB and Storage, Gemini, shared package packages/core (@stashmind/core). Gates: typecheck with 0 errors, npm run verify green, 43 tests or more, plus test:live wherever credentials exist.
- EasyTutor (github.com/dmuhoro/easytutor): Expo SDK 55 and React Native, Supabase with pgvector, Zustand, NativeWind, Zod, Anthropic/Groq/Ollama model router, portals high_school (KICD), university, knowledge_explorer. Gates: typecheck 0 errors, lint 0 errors, 181 tests or more.
- Keep the repositories separate. EasyTutor changes are committed only to the EasyTutor repo and StashMind changes only to the StashMind repo. Cross-repo coupling happens only through a versioned package and an HTTP API. Never copy source between repos.
- Never merge to main. Work on a branch named merge/stashmind-integration in each repo and open pull requests. Never force-push. Never rewrite pushed history.
- No change in behaviour for existing users. Every new capability sits behind a feature flag that defaults to OFF.
- Gate rule: a repo's gates are green before and after every commit in that repo. Test counts may only go up. Never weaken, skip or delete a test, lint rule or type check.
- EasyTutor non-negotiable: online mode (Anthropic/Groq) and offline mode (local Ollama) both keep working.
- Secrets: StashMind service credentials exist only on the EasyTutor server side. Never in the mobile bundle, repos, logs or reports.

SETTINGS
MODE: UNATTENDED. Stop only for a human gate that blocks the next layer's code, three failed attempts at the same gate, or completion. If a gate only blocks verification, build against fakes, mark the item DONE-PENDING-GATE, and continue.

STATE AND CONVENTIONS
Follow Software Factory's Constitution, ADR numbering, AGENTS.md and sprint-file format. Each layer has a sprint file (goal, tasks, acceptance, evidence). Write run state to runs/<id>/STATE.md after every layer so the run can resume.

TRUTH RULES
1. Every earlier report of completion, including those in this task and your own, is an unverified claim.
2. Statuses: VERIFIED (executed and passed, with the command and output), IMPLEMENTED-UNTESTED, PARTIAL, STUB-OR-FAKE, MISSING, BROKEN, DONE-PENDING-GATE, UNVERIFIABLE.
3. Forbidden: dummy keys, silent fallbacks that hide failures, simulated
```

```
dispatch, hard-coded return values, mocks on a production path, tests that
mock the thing under test, catching an error and reporting success.
4. Every feature layer adds tests. Report the test count before and after in
each repo.
5. Docs must match code. Never write production-ready or complete without
evidence.
6. No secrets in anything you write.

STAGE M0: VERIFY BEFORE BUILDING (hard gate)
1. Clone both repos. Record commit SHAs, branches, tags and open pull
requests. State whether StashMind has the tag v1.0.0-rc1 and EasyTutor
v1.0.1. List which StashMind layers C00 to C23 exist as one commit each and
which were pushed straight to main. Record every deviation from the agreed
process.
2. StashMind: run npm ci, npm run verify and bash stashmind-doctor.sh --boot.
Run test:live and the Playwright end-to-end suite if they exist and
credentials are available. Count tests (reported: 43) and list which features
have none. Check that production refuses to start without configuration, that
model IDs are checked against Google's live model list, that every
unauthenticated request is rejected, and that two-user isolation tests exist
and pass. For Daraja, Telegram, Swahili, voice and legal items, report
whether each is DONE-PENDING-GATE or fake.
3. Deployed StashMind (https://stashmind-nu.vercel.app): run the smoke
checks. With a test account, create a real lesson from a real PDF; record the
time and result. Establish where jobs run on this host, what its time limits
are, and whether ffmpeg audio assembly works there. If long jobs cannot run
on this host, record an S0 finding and propose a container host.
4. EasyTutor: run typecheck, lint and the tests (reported: 181). Check that
the three September P0 gaps (Ollama /v1 routing, feedback tables, free-form
roadmap refresh) are closed and covered by tests. Run the offline-mode test.
Check the deployed URL and the mobile build status.
5. Compare README, ADR and status-document claims with the code in both
repos. Fix wrong documents in their own layer.
6. Write the M0 report as a table of claim, evidence and status in each repo.
If any S0 or S1 finding exists, create a stabilization sprint in the owning
repo and fix it first, one gated layer commit at a time. If it cannot be
fixed without a human gate, stop and report.

STAGE M1: ARCHITECTURE DECISIONS
Write these ADRs in the repo that owns each decision and cross-link them. Use
the defaults unless you document a better choice with evidence.
- ADR-M1 Integration pattern: shared core package plus typed API client. No
database merge and no data migration. StashMind owns lessons, cards, audio
and ask-the-source threads. EasyTutor owns learner profiles, curriculum and
roadmaps, the pgvector RAG corpus and progress.
- ADR-M2 Identity: EasyTutor is a workspace in StashMind with a service
```

account. A mapping of (workspace, externalUserId) to a StashMind user is stored in StashMind. EasyTutor's server calls StashMind on behalf of the learner. The mobile app never holds a StashMind key. Learners get no second login.
- ADR-M3 Offline and degradation: online uses StashMind for audio lessons, cards and ask-the-source. Offline or during an outage, EasyTutor uses local Ollama with the core prompts and schemas for cards and quizzes, and device text-to-speech. Dual-host audio is online-only.
- ADR-M4 Packaging across separate repos: @stashmind/core is published as a versioned package (private registry, or a tagged git dependency if no registry token exists). It must be React Native safe. @stashmind/client is generated from OpenAPI. Semver and changelogs for both.
- ADR-M5 Model-router convergence: a ModelProvider interface in core, conformance tests, adapters for Gemini, Anthropic, Groq and Ollama. Each repo adopts it behind a flag after parity is proven. Duplicates are removed only in a later sprint.
- ADR-M6 Portal and preset mapping: high_school (KICD) maps to student, with KICD alignment remaining EasyTutor content; university maps to student; knowledge_explorer maps to learner; professional is an optional extra track. The mapping lives in core config with tests.
- ADR-M7 Rollout: flags default OFF, staged rollout (internal, small percentage, everyone), rollback by flag.
- ADR-M8 Cross-system privacy and observability: correlation IDs across both systems, updated list of data processors, consent, deletion propagation.

LAYERS (one commit per layer, in the repo that owns the change)
M2 CORE HARDENING (StashMind). Build packages/core to ESM, CJS and types with an exports map. Make it React Native and Metro safe: no Node-only modules, no process.env or Buffer assumptions. Tests, README, version 1.1.0 and changelog. Publish (gate H2) or tag. Acceptance: a clean EasyTutor checkout installs it and Metro bundles it.
M3 CLIENT AND ON-BEHALF-OF (StashMind). OpenAPI-generated @stashmind/client. Workspace service accounts. External-identity mapping and an on-behalf-of mechanism. Per-workspace quotas and rate limits. Audit log. Deletion-propagation endpoint. Backward compatible. Acceptance: contract tests and a two-workspace isolation test against real Appwrite.
M4 EASYTUTOR BRIDGE (EasyTutor). A server-side bridge in EasyTutor's existing backend that holds the StashMind credentials. A Supabase link table (stashmind_links) with row-level security and a migration. Timeouts, retries, a circuit breaker and error mapping. Quotas aligned with EasyTutor plans. Flag stashmind_enabled. Acceptance: tests against a StashMind contract stub, plus a live staging test (gate H3).
M5 EASYTUTOR UI (EasyTutor). An Audio Lessons module behind the flag: create from text, URL or PDF; progress; player with audio, cards and transcript; ask-the-source; practice; saved cards; offline downloads; data-saver. Portal-aware presets. Accessibility. Zod validation of every response using core

schemas. Follow the repo's Zustand and NativeWind conventions. Acceptance: unit and component tests plus one end-to-end path.
M6 OFFLINE AND DEGRADE (EasyTutor). Ollama card and quiz generation using the core prompts and schemas. Device text-to-speech. An offline queue for lesson requests. Clear states for offline and outage. Acceptance: tests simulating airplane mode and a StashMind outage, and EasyTutor's existing offline tests still pass unchanged.
M7 ROUTER CONVERGENCE (core and both repos). ModelProvider interface, four adapters, conformance suite. StashMind adopts behind a flag. EasyTutor adopts behind a flag only after parity on shared fixtures (same outputs within tolerance, token accounting). The existing 181 tests stay untouched and green.
M8 LEARNING-DATA LINKS (stretch). Lesson to roadmap-node links. Idempotent saved-card sync into EasyTutor's review. Lesson summaries into pgvector RAG only with explicit user consent and deletion propagation.
M9 CROSS-PRODUCT HARDENING. End-to-end scenarios: create a lesson in EasyTutor, audio plays, save a card, ask a question, go offline, exceed a quota, StashMind outage, recovery. Scan the built mobile bundle for secrets. Threat model of the bridge. Tests that user A cannot see user B's lessons through the bridge. Updated privacy documents and processor lists in both repos. A load test of the bridge. Correlation IDs.
M10 RELEASE. Versions (core and StashMind 1.1.0, EasyTutor 1.1.0), changelogs, rollout plan, rollback runbook, docs in both repos, staging smoke test. Open the pull requests. Do not merge.

COMMIT PROTOCOL
- One commit per layer per repo. A layer that spans both repos produces one commit in each.
- Format: type(layer-MNN-slug): imperative subject, 72 characters or fewer. Body sections: Why, What, Evidence (exact commands and results), Tests (added N, total M), Gates. Trailers: Merge-Layer: MNN, and Counterpart: <repo>@<sha> when the layer touches both repos.
- Run the repo's gates before every commit. Never commit red. Push after every layer. Never force-push or amend pushed commits. Fix later defects with a new fix(layer-MNN-slug) commit.
- Each layer commit includes its own sprint-file evidence and matrix updates. No status-only commits.

HUMAN GATES
H1 ADR review (retroactive in unattended mode). H2 registry token or approval of a tagged git install. H3 staging service-account key and EasyTutor staging settings. H4 real-phone and listening tests. H5 store release decisions. H6 privacy sign-off on cross-system data flows.

FAILURE PROTOCOL
Maximum three repair attempts per failing gate. Then write BLOCKERS.md in the

```
owning repo, commit nothing broken, stop dependent layers, and continue
independent ones.

NO SELF-CERTIFICATION
After M10, do not declare the job verified. Produce a handoff for a DIFFERENT
agent: a status matrix per repo and a clean-room reproduction script that re-
runs every claim from a fresh clone. The job is only verified when that
independent re-audit finds no false VERIFIED statuses.

SOFTWARE FACTORY SCORECARD (required: runs/<id>/SCORECARD.md)
Record: layers planned versus completed; first-time gate pass rate; commits
per day; evidence completeness (share of claims backed by command output);
human interventions and the reason for each; unattended run time; resumptions
after interruption; test counts before and after in both repos; any weakened,
skipped or deleted check (must be zero); gaps between your reports and the
repos found by the independent re-audit (must be zero false VERIFIED); cost
or tokens per layer where available.

DEFINITION OF DONE
1. Both repos' gates are green with test counts at or above 43 and 181.
2. With every flag OFF, nothing changes for existing users and EasyTutor's
existing end-to-end tests pass unchanged.
3. With the flag ON, the cross-product end-to-end scenario passes on staging.
4. No secrets appear in the mobile bundle, repos or logs.
5. EasyTutor's offline mode works.
6. Pull requests are open in both repos and not merged.
7. The independent re-audit finds zero false VERIFIED statuses.
8. The scorecard is complete.

START
Begin with Stage M0 now. State your environment and capabilities first:
shell, git, network, Docker, and access to both repositories.
```

---

4. What 'Software Factory passed this test' should mean

The job is a fair test of Software Factory if it meets all of these:

- **It found the problems itself.** Stage M0 produces real findings in at least one of the repos before any building starts.
- **It stayed inside the rules.** Zero weakened or skipped checks, zero merges to main, zero force-pushes, flags default off.
- **Its claims survive an outside check.** A different agent's clean-room re-audit finds no false VERIFIED statuses.

- **It needed few humans.** Human interventions are limited to the listed gates, and the scorecard records each one.
- **It recovered.** If it was interrupted, it resumed from its state file without redoing or corrupting finished layers.

Bring back to me: the M0 report, the scorecard, `git log --oneline` for each repo, and the re-audit output. I will compare them and tell you where the story and the evidence differ.